



Borijan Georgievski



Summary

The article presents an effective one-shot learning approach for face recognition using Siamese Networks and a pretrained FaceNet model, enabling real-time employee recognition with high accuracy despite a limited dataset.

Abstract

The article discusses the challenges and solutions for face recognition with limited data, specifically when only one image per person is available. It highlights the use of Siamese Networks and the FaceNet architecture, which leverages triplet loss for training, to overcome the limitations of standard Convolutional Neural Networks (CNNs) in one-shot learning scenarios. The FaceNet model, pretrained on large datasets like CASIA-WebFace and VGGFace2, is fine-tuned for specific applications, such as recognizing company employees. Additionally, the Multi-task CNN (MTCNN) framework is employed for face detection and alignment, which is crucial for minimizing noise and focusing on the face region. The article also details the development of a Python application, web and mobile apps, and a REST web service for real-time face recognition, demonstrating the practical implementation of these advanced techniques in a corporate setting.

Opinions

- The authors express that large-scale public datasets for face recognition are lacking, which has limited advancements in the field to large companies like Google and Facebook.



by researchers as a significant contribution.

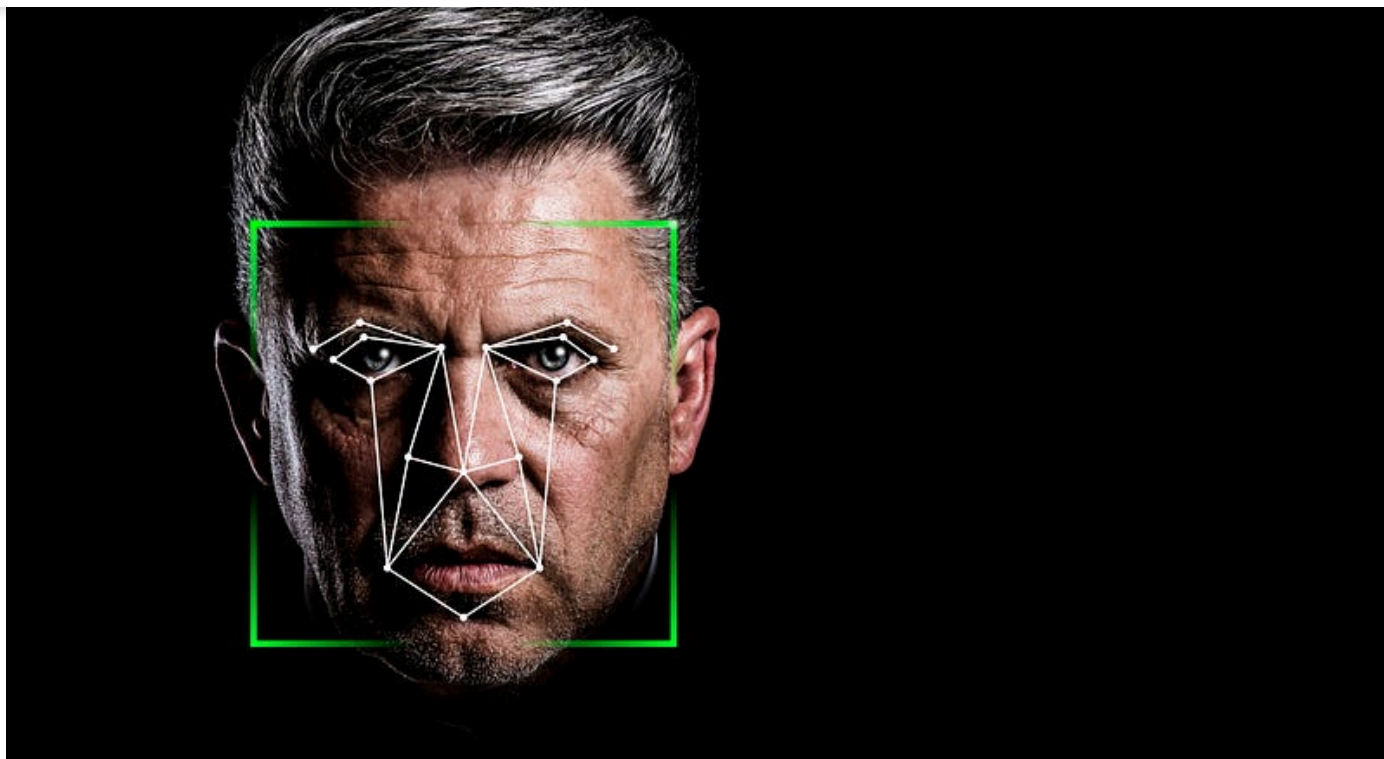
- The article conveys that standard CNNs are not suitable for one-shot learning due to their reliance on large amounts of data and the impracticality of retraining for each new class.
- The authors consider the triplet loss function as a key component in training Siamese Networks effectively, particularly when dealing with a vast number of images.
- They emphasize the efficiency of selecting hard triplets for training, as opposed to all possible combinations, to improve model convergence.
- The authors suggest that the MTCNN framework is superior to other methods, such as the Vision library, for face detection tasks in real-

.. ..



Use the OpenAI o1 models for free at
OpenAIo1.net (10 times a day for free)!

Face Recognition with One-Shot Learning



© Jack Moreh — Facial Recognition Concept

This article demonstrates a very effective approach for face recognition when the dataset is very limited. Using only one image per person (one-shot learning), we managed to create a highly accurate model for recognizing company employees in real-time.

Convolutional Neural Networks (CNNs) have taken the computer vision community by storm, significantly surpassing the state-of-the-art techniques in many applications. One of the most important ingredients for success of such methods is the availability of training data. The annual ImageNet Large Scale Visual Recognition Challenge (ILSVRC) competition, active since 2010, was instrumental in providing data for general image classification tasks. The ILSVRC2017 image dataset contains approximately two million images, having two separate datasets for object localization and object detection. More

However, in the world of face recognition, large scale public datasets have been lacking, and largely due to this factor, most of the recent advances in the community remain restricted to Internet giants such as Facebook and Google. For example, the most recent face recognition method by Google was trained using 260 million images. The size of this dataset is almost three orders of magnitude larger than any publicly available face dataset. Needless to say, building a dataset this large is beyond the capabilities of most research groups, particularly in academia.

WE'VE DECIDED
TO TAKE BIG
DATA TO THE
NEXT LEVEL...



**HUMONGOUS
DATA**

© D. Fletcher for CloudTweaks.com

Despite the lack of collaboration from the giants like Google and Facebook, many researchers around the world make great efforts in collecting new public

consisting of more than 500K faces and 10K classes gave ML enthusiasts the opportunity to actually implement state-of-the-art algorithms. Moreover, being able to train the model also means being able to share it as a **pretrained network** by saving all weights after the training phase.

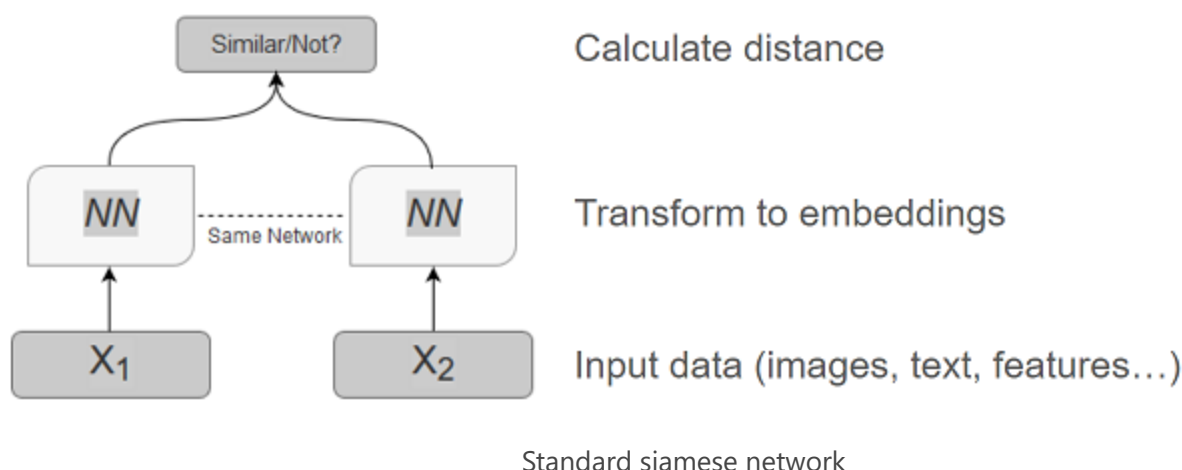
Siamese Networks and FaceNet

Having to work with a small dataset (one image per class, 440 classes) greatly limits the number of applicable techniques. It seems that the standard CNNs have big problems with one-shot learning tasks, mainly because of:

- Standard CNNs work phenomenally when they are fed large amounts of data. However, they cannot find patterns specific to a certain class if there is not enough training data for that class.
- It is certainly not convenient to retrain the model every time we add a picture of a new person to the system. Training CNNs takes a lot of computational power and time. So if we want to recognize employees, we do not want to train a new network every time we have a new employee, or if someone leaves the company.

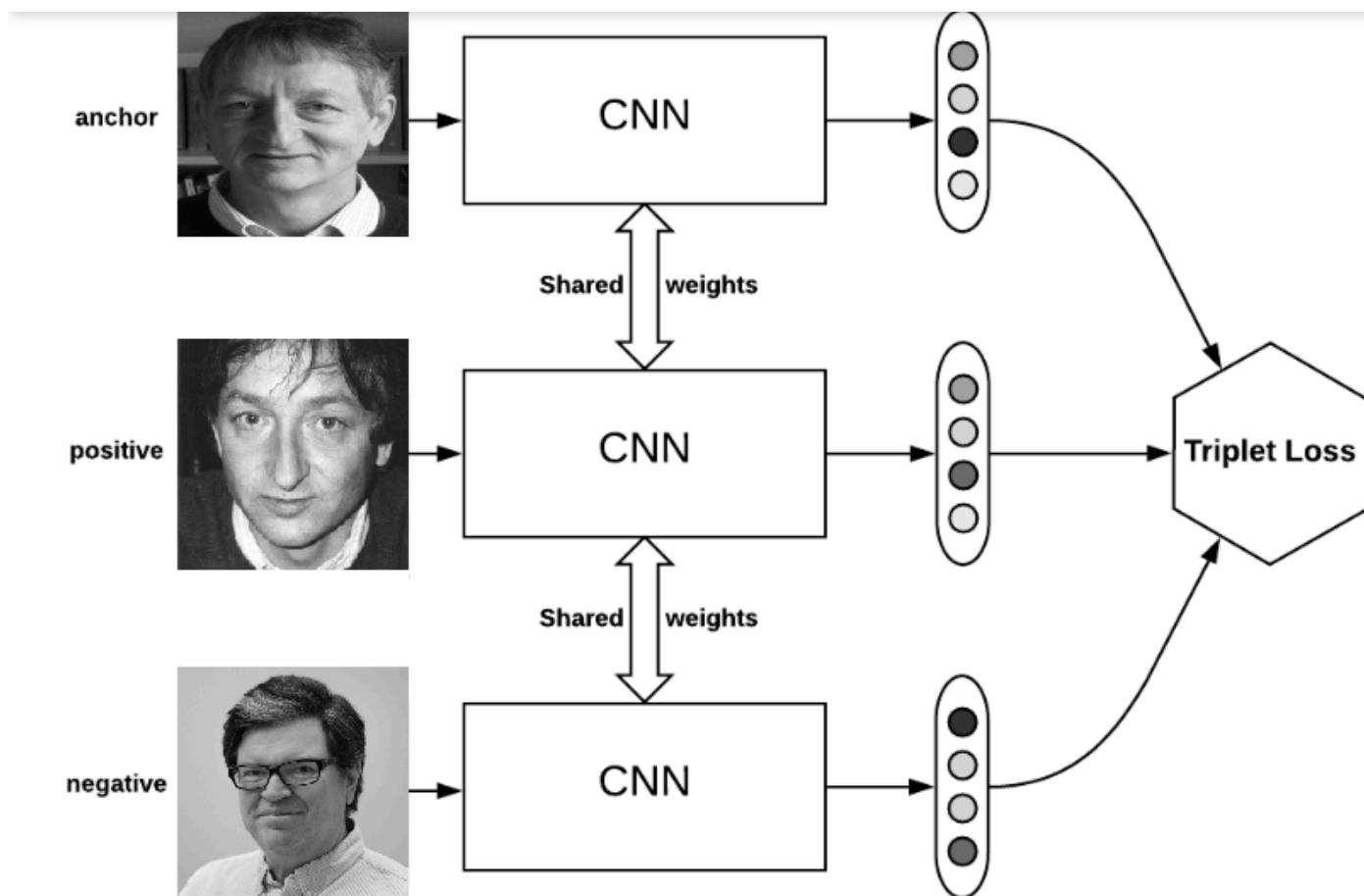
One of the best deep learning architectures that work great with one-shot learning is called Siamese Network. The idea is quite simple:

- Take an input and extract its **embedding** (mapping to a vector of continuous numbers) by passing it through a neural network.
- Repeat step 1 with a different input.
- Compare the two embeddings to check whether there is a similarity between the two data points. These two embeddings act as a latent



Very often, learning in siamese networks is done by the **triplet loss** function. Such models require three input images on training phase, and the loss is calculated as:

- Extract embeddings from an **anchor** input image a .
- Extract embeddings from a **positive** input image p (same class as the anchor).
- Extract embeddings from a **negative** input image n (different class from the anchor).
- Calculate the Euclidean distances $d(a, p)$ and $d(a, n)$. Ideally, the first distance should be as small as possible, while the latter should be as big as possible.
- The loss function is defined as: $L = \max(d(a, p) - d(a, n) + \alpha, 0)$, where α is a parameter that defines how far away the dissimilarities should be, and enforces a distinction between the positive and the negative image.



Triplet loss on two positive classes (Geoffrey Hinton) and one negative class (Yann LeCun)

FaceNet is a siamese network architecture developed by Google in 2015. They use a deep CNN to directly optimize the embedding itself, rather than an intermediate bottleneck layer as seen in previous approaches. Google tried different types of architectures for FaceNet, and the most successful type was based on the GoogLeNet style Inception models. One such model contains around 7M parameters, and is trained on up to 260M images. All of the FaceNet models are trained using the triplet loss function.

Considering that the model needs three images for the triplet loss for each forward pass, even Google would face difficulty to run all possible triplet combinations out of 260 million images. Additionally, generating all possible



Translate to

resulting in slower convergence of the model. Therefore, Google chose a smarter approach: for a given anchor image a , they use triplets that violate the triplet constraint, by selecting the inputs p and n such that $d(a, p)$ is maximized, and $d(a, n)$ is minimized.

Knowing how a state-of-the-art model works is great, but we also need the model to be trained on large amounts of data so it can be useful. Thankfully, [David Sandberg has released pretrained models](#) based on FaceNet that use the CASIA-WebFace (~453K faces and ~10K classes) and VGGFace2 (~3.3M faces and ~9000 classes) face recognition datasets. We ended up fine-tuning the model trained on VGGFace2 for our needs.

Multi-task CNNs for Face Detection

Face Recognition has greatly improved over the last few years as a result of new deep learning architectures. But before we can perform classification, we need to localize (or detect) the object of interest. This is mainly because we want to minimize the “noise” that exists outside the person’s face in an image. We made use of the [Multi-task CNN framework for face detection and alignment \(MTCNN\)](#). The framework, as suggested by its name, is used to compute multiple tasks:

- **Face classification:** the first objective is a two-class classification problem (whether or not there exists a face in the window). It uses cross-entropy loss where p_i is the probability produced by the network that indicates sample x_i being a face:

- **Bounding box regression:** for each candidate window, we predict the offset between that window and the nearest ground truth. The ground truth is a human-labeled window which is represented with four values: x and y coordinates of the top left point of the window, window height and window width. The loss is the Euclidean distance for each sample x_i :

$$L_i^{box} = \|\hat{y}_i^{box} - y_i^{box}\|_2^2$$

- **Facial landmark localization:** similar to the bounding regression task, facial landmark detection is formulated as a regression problem. Five facial landmarks are detected: (x, y) coordinates of the left eye, right eye, nose, left mouth corner, and right mouth corner. The loss is defined as:

$$L_i^{landmark} = \|\hat{y}_i^{landmark} - y_i^{landmark}\|_2^2$$

The framework consists of three stages of carefully designed deep CNNs to predict face and landmark location:

- Proposal Network (**P-Net**) is introduced to find all the possible windowed candidates. By using regression instead of classification, it tries to find the bounding box vectors, and then it calibrates the image based on the particular estimated bounding boxes. Afterwards, it uses Non-maximum suppression (NMS) to merge highly overlapped

- All candidates are fed to a second network called the Refine Network (**R-Net**). Its goal is to further reject a large number of false candidates, by performing additional calibration with bounding box regression, and NMS.
- The final stage is similar to the previous stage. Output Network (**O-Net**) is used to identify face regions with stricter thresholds, and to output the five common facial landmarks' positions, which were mentioned above.

The Big Picture

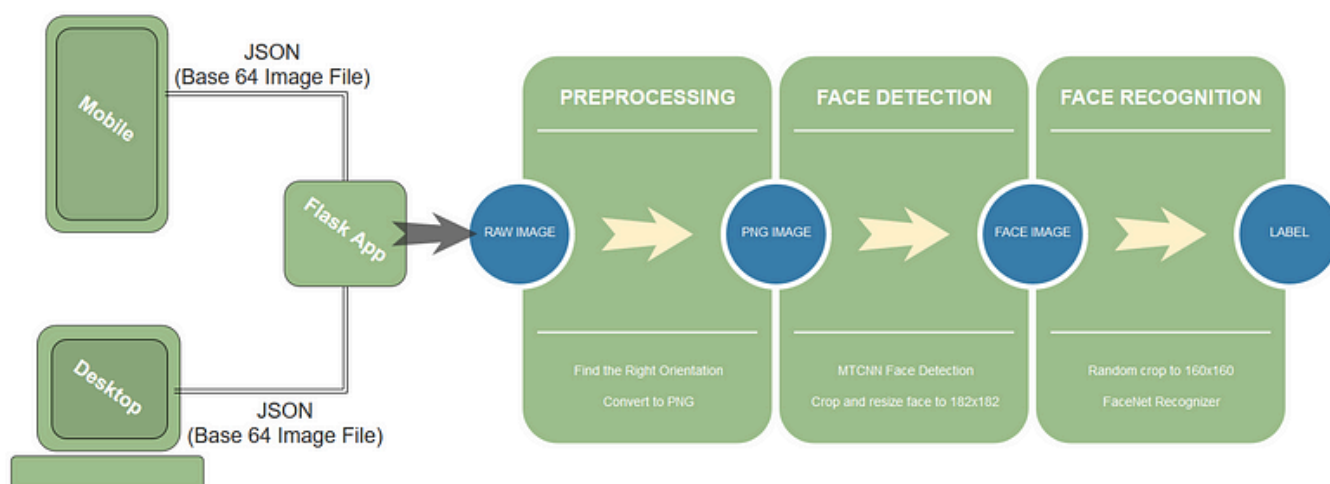
Having collected all of the puzzle pieces, we created a Python application for training the face recognition model, and for classifying new images. The pretrained FaceNet model is used as a **feature extractor**, whose output is fed into a simple classifier (KNN, one nearest neighbor) that returns the final prediction.

Our training dataset consists of one image per class (Netcetera employee), for 440 classes, while the test dataset consists of 5 to 10 images per class, for 78 classes. Out of every tested simple classifier, the following obtained the best results:



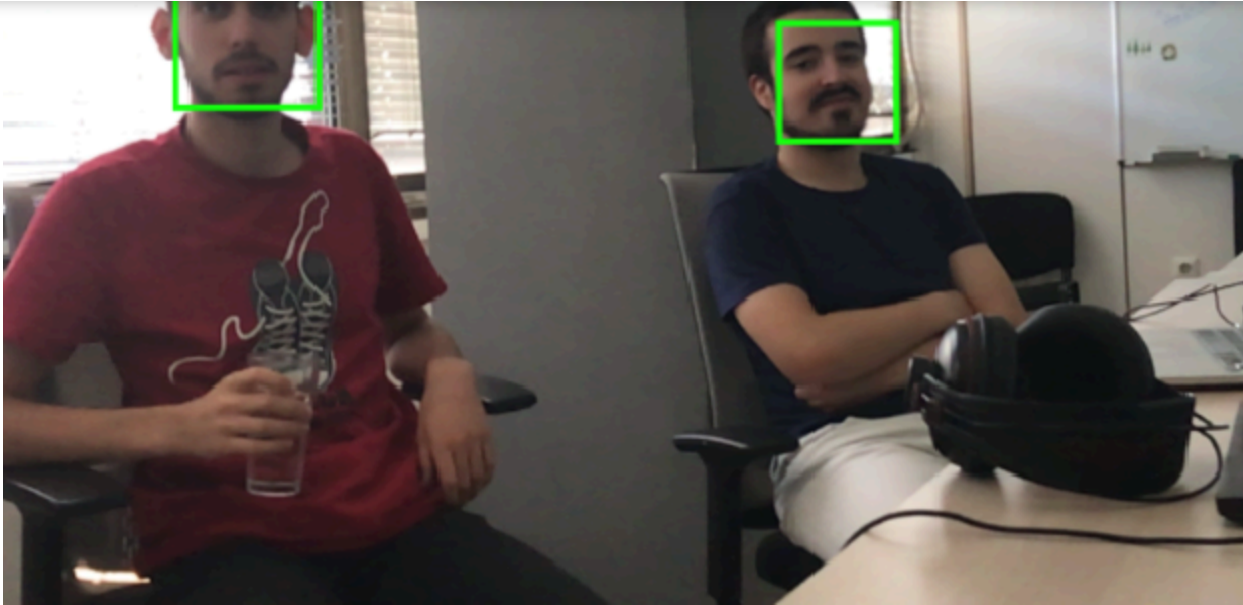
KNN (N = 1)	0.951	0.016s
Naive Bayes	0.951	0.047s
Logistic Regression	0.918	0.016s
Boosting (KNN + NB + LR)	0.951	0.078s

We also developed web and mobile (iOS) apps that communicate with a REST web service, sending raw image data and receiving the predicted name of the employee in the given image.



The classification pipeline

The real-time video iOS application makes use of the Vision library to have a real-time tracking natively on the phone. The tracking makes corrections of the face coordinates every 0.1 seconds. Meanwhile, it asynchronously sends the detected faces to the REST API, in order to get the names of the employees, and show them on the screen. Note that the Vision library also has a face detection feature, but in practice it performs much worse than MTCNN, so we settled with performing the final face detection on the server side.



Screenshot of the real-time video application

Thanks for reading!

Machine Learning

Deep Learning

Computer Vision

Face Recognition

One Shot Learning

Recommended from ReadMedium



Alexander Nguyen

The resume that got a software engineer a \$300,000 job at Google.

1-page. Well-formatted.

4 min read



Hamza Asif



Translate to

There are always some things that we think are difficult to understand but in reality, we are not looking at those things from the right...

18 min read



Sunny Kumar

Object tracking in YOLO using python and open cv

Yolo implementation of object tracking in python. Computer vision object tracking. open cv realtime object tracking using yolo and python3.

4 min read



Happy Verma

What are the Features of WebRTC in Making Real-Time Communication App Development

Web Real-Time Communication (WebRTC) has revolutionized the way we communicate online, enabling real-time exchange of video, audio, and...

3 min read



Juneta Tao

Image Based Virtual Try-On (VITON)

The goal of image based VITON is to fit a given cloth image to a person in a reference image. The majority methods include two steps. First...

5 min read



Amos Stailey-Young



Translate to

9 min read



Free OpenAI o1 chat Try OpenAI o1 API